

Facultad de Ciencias UNAM

Estructuras de Datos

Nota4: Cota superior asintótica: Notación O (o mayúscula)

Profesora: Virginia Teodosio Procopio

Ayudantes: Leonardo Gallo Guerrero

José Manuel Madrigal Ramírez

23 de Septiembre de 2026

Notación O

Considerando el hecho de que el grado del polinomio que define una función de complejidad no crece al considerar más o menos operaciones básicas, siempre que no descartemos la operación esencial, hay que notar algo sobre el crecimiento de las funciones en sí mismas.

Supongamos que tenemos dos algoritmos A y B cuyas funciones de complejidad son $f(n) = 2n$ y $g(n) = n^2$, no hace falta mayor análisis para ver que para toda $n \in \mathbb{Z}^+$ tal que $n > 2$ siempre se cumple que:

$$f(n) < g(n)$$

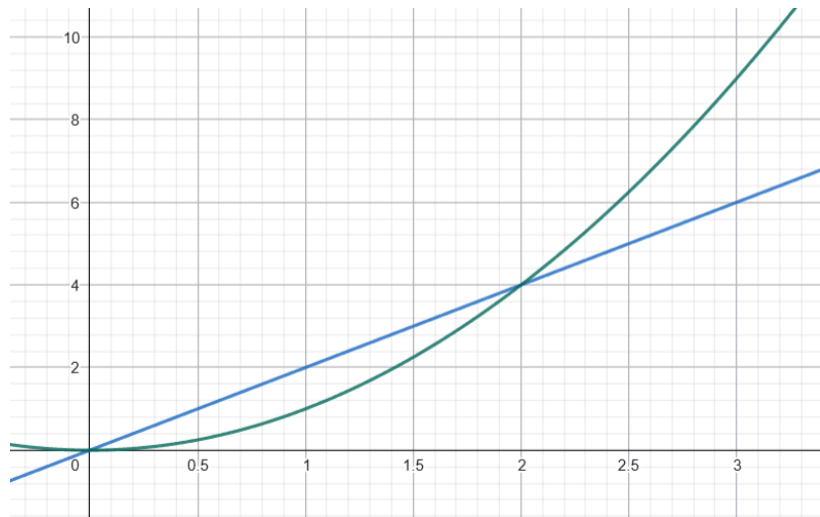


Figura 1: Gráficas de las funciones $f(n) = 2n$ y $g(n) = n^2$

Sin embargo, ¿qué sucede si $f(n) = 1000n + 1000000$?

Como muestra la siguiente gráfica, parece que $f(n)$ permanece por arriba de $g(n)$.

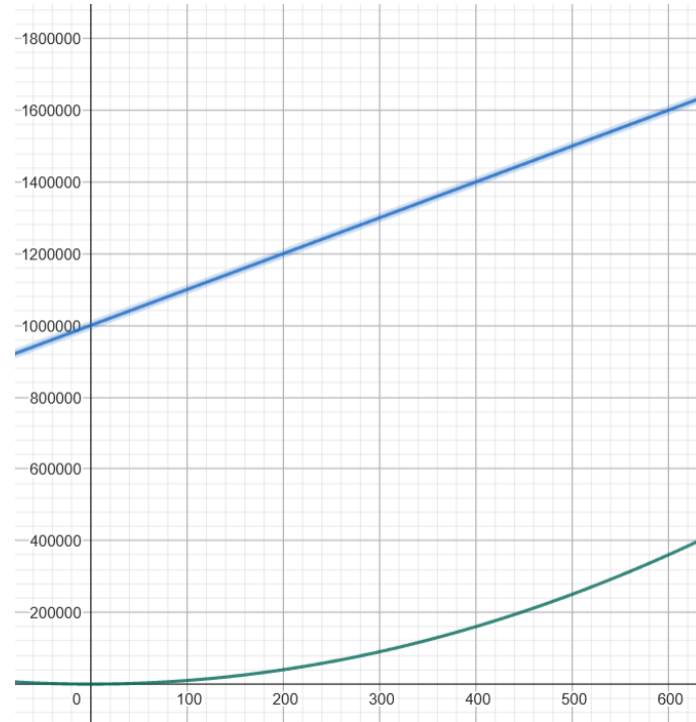


Figura 2: Gráficas de las funciones $f(n) = 1000n + 1000000$ y $g(n) = n^2$

Sin embargo, inevitablemente llega el momento, cuando $n \geq 1619$, y entonces $f(n) < g(n)$.

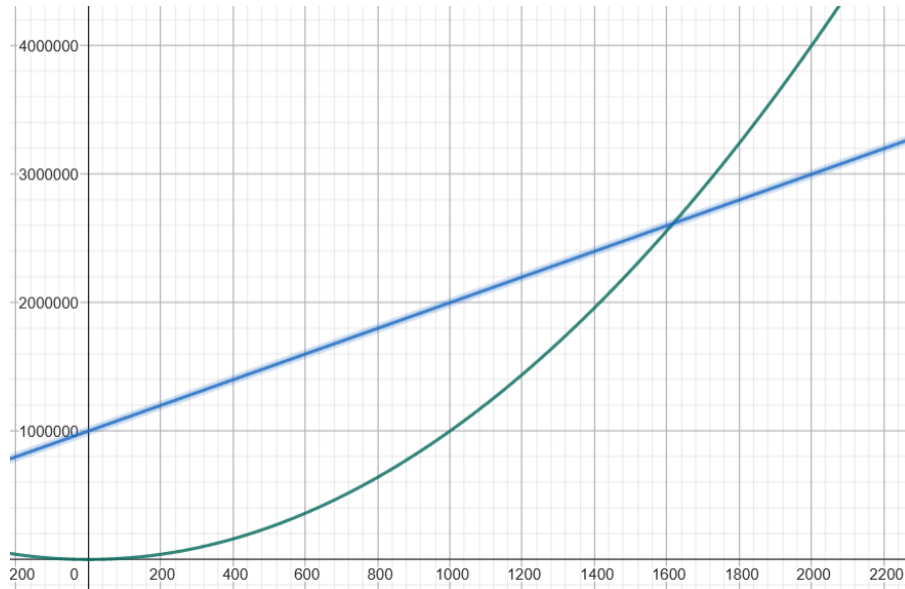


Figura 3: Gráficas reescaladas de las funciones $f(n) = 1000n + 1000000$ y $g(n) = n^2$

Por lo visto parece que todo algoritmo que tenga una función de complejidad definida por un polinomio cuadrático, dada una complejidad de entrada n suficientemente grande, terminará por superar en el uso de recursos a cualquier algoritmo que tenga una función de complejidad dada por una función lineal.

Podemos vernos tentados a escribir $f(n) \leq g(n)$ pero debemos ser cuidadosos pues ese enunciado declara que la función f es menor a la función g para todo n .

Entonces necesitamos indicar que $g(n)$ sólo es mayor que $f(n)$ a partir de cierto tamaño de

entrada específico. Además queremos que los polinomios de primer grado queden en el mismo conjunto pues nos interesa saber que un algoritmo no vaya a “explotar” en el uso de recursos a partir de cierta complejidad de entrada. Entonces una función, digamos $h(n) = 8n + 3$, debería quedar en el mismo saco que $f(n) = 1000n + 1000000$, que si bien tienen gráficas con pendientes muy distintas, su crecimiento nunca “explota” y es predecible para una n dada.

Este par de requerimientos se formalizan mediante la siguiente definición:

Definición 1.1 (Orden de complejidad asintótica). Decimos que f pertenece al orden de g , denotado como $f = O(g)$ o bien $f \in O(g)$, si y sólo si existe $N \in \mathbb{N}$ y $c \in \mathbb{R}^+$ tales que para toda $n \in \mathbb{N}$ que cumpla $n > N$ sea cierto que:

$$f(n) \leq cg(n).$$

Retomando el ejemplo (y cambiando el nombre de la variable independiente de n a m para evitar confusiones) de $f(m) = 1000m + 1000000$, podemos demostrar que pertenece al orden $O(n)$ si exhibimos un número natural N a partir del cual nuestra función f , evaluada en cualquier m mayor a N , quede por debajo de un múltiplo de la función n . Es decir, encontrando un natural N y una constante c en los reales positivos tales que:

$$f(m) \leq cn.$$

Con certeza sabemos que

$$1000m + 1000000 < 1000m + 1000000m \quad \text{para todo } m \geq 1.$$

Entonces, como

$$f(m) = 1000m + 1000000 < 1001000m,$$

podemos asegurar que hemos encontrado un $c = 1001000$ y un $N = 1$ para los cuales se cumple la definición de orden asintótico y entonces $f(m) = O(n)$.

Podríamos ser más diligentes y buscar otra constante c más pequeña pero la condición de nuestra definición únicamente nos pide encontrar algún c y algún N que hagan verdadera la desigualdad $f(n) \leq cg(n)$ para todo $n > N$.

Del mismo modo podremos encontrar el orden de las funciones de complejidad de aquellos pequeños algoritmos que designamos en la *Nota 3* como **Ejemplo1**, **Ejemplo2** y **Ejemplo3**:

Habíamos establecido que la función de complejidad para **Ejemplo1** era:

$$T(n) = 6n + 4.$$

Tenemos que la siguiente desigualdad se cumple para todo $n > 1$:

$$6n + 4 \leq 6n + 4n = 10n.$$

Puede ser el caso que tanta n cause confusión. Cuando escribimos $O(n)$ nos referimos al orden lineal y n representa la función identidad $i(n)$ (aquella cuya gráfica es una recta con pendiente 1 y que pasa por el origen del plano cartesiano). Bajo esta consideración y por la desigualdad anterior tenemos que $6n + 4 \leq 10i(n)$ para todo $n > 1$. Dicho de otra forma: el múltiplo 10 de la función identidad acota asintóticamente a la función de complejidad $T(n) = 6n + 4$, por lo que podemos deducir que $T(n) \in O(i) = O(n)$. **La notación común para referirse al orden de las funciones lineales es $O(n)$.**

Cuando dedujimos una segunda función de complejidad para el **Ejemplo1** considerando únicamente la operación esencial obtuvimos:

$$T(n) = 2n.$$

Y dado que $2n \leq 2n$, tenemos que para $c = 2$ y $N = 0$, $T(n)$ queda acotada superiormente por la función identidad, es decir, tenemos nuevamente que $T(n) \in O(n)$.

Antes de continuar, cabe aclarar que la notación $O(g)$ se refiere a un conjunto de funciones. La condición de pertenencia a ese conjunto es la que dicta la definición, es decir, una función de complejidad f pertenece al conjunto $O(g)$ si un múltiplo de la función g la acota asintóticamente por arriba a partir de un elemento N del dominio de f . A partir de este hecho se derivan varias propiedades del conjunto $O(g)$ (véase [1]):

Sean f, f_1, f_2, h y g funciones no negativas se cumple:

1. **Autocontención:** $g \in O(g)$.
2. **Subconjunto:** Si $f \in O(g) \Rightarrow O(f) \subset O(g)$.
3. **Identidad:** $O(f) = O(g) \iff f \in O(g)$ y $g \in O(f)$. *Nota:* no implica que $f = g$.
4. **Transitividad:** si $f \in O(g)$ y $g \in O(h)$, entonces $f \in O(h)$.
5. **El menor:** si $f \in O(g)$ y $f \in O(h)$, entonces $f \in O(\min(g, h))$.

6. **Regla de la suma:**

Si $f_1 \in O(g)$ y $f_2 \in O(h)$, entonces

$$f_1 + f_2 \in O(\max(g, h)).$$

7. **Regla del producto:**

Si $f_1 \in O(g)$ y $f_2 \in O(h)$, entonces

$$f_1 \cdot f_2 \in O(g \cdot h).$$

8. **Criterio para ordenar:**

Sea

$$\limsup_{n \rightarrow \infty} \frac{f(n)}{g(n)} = k$$

tenemos:

- a) **Prueba de identidad:** si $k \neq 0$ y $|k| < \infty$, entonces $O(f) = O(g)$.
- b) **Prueba de inclusión:** si $k = 0$, entonces $f \in O(g)$, es decir,

$$O(f) \subset O(g),$$

además se verifica que $g \notin O(f)$.

- c) **Prueba de exclusión:** si $k = \pm\infty$, entonces $f \notin O(g)$, es decir,

$$O(f) \supset O(g),$$

además se verifica que $g \in O(f)$.

Demostrando las propiedades de suma y producto

Demostraremos la regla 6 y 7 que nos serán muy útiles al establecer el orden de las funciones de complejidad de nuestros algoritmos. La estrategia para demostrarlas es similar para ambas propiedades: A partir de las premisas nombramos las constantes que se siguen de la definición. Proponemos un n_0 que nos permita enunciar simultáneamente las desigualdades de funciones y, ya sea el caso, sumamos o multiplicamos ambas igualdades para llegar al resultado esperado. Veámoslo con más detalle a continuación.

Demostración de la regla de la suma

Teorema 1.1 (Regla de la Suma). Si $f_1 \in O(g)$ y $f_2 \in O(h)$, entonces $f_1 + f_2 \in O(\max(g, h))$.

Demostración. A partir de $f_1 \in O(g)$ y $f_2 \in O(h)$, que son nuestras premisas, podemos deducir, por la definición de O , que existen $c_1 \in \mathbb{R}^+$ y $n_1 \in \mathbb{N}$ tales que para todo $n > n_1$ se cumple que

$$f_1(n) \leq c_1 g(n).$$

También de las premisas podemos deducir la existencia de $c_2 \in \mathbb{R}^+$ y $n_2 \in \mathbb{N}$ tales que para todo natural $n > n_2$ se cumple que

$$f_2(n) \leq c_2 h(n).$$

Sea

$$n_0 = \max(n_2, n_1).$$

A partir de lo dicho en el párrafo anterior, podemos decir que para todo natural $n > n_0$ se cumple simultáneamente que:

$$f_1(n) \leq c_1 g(n)$$

$$f_2(n) \leq c_2 h(n)$$

Si sumamos ambas desigualdades obtenemos que:

$$\begin{aligned} f_1(n) + f_2(n) &\leq c_1 g(n) + c_2 h(n) \\ &\leq c_1 \max(g(n), h(n)) + c_2 \max(g(n), h(n)) \\ &= (c_1 + c_2) \max(g(n), h(n)). \end{aligned}$$

Así,

$$f_1(n) + f_2(n) \leq (c_1 + c_2) \max(g(n), h(n)) \quad \text{para todo } n > n_0.$$

Por la definición de O , podemos concluir que

$$f_1 + f_2 \in O(\max(g, h)).$$

□

Análogamente podemos probar la regla del producto:

Demostración de la regla del producto

Teorema 1.2 (Regla del producto). Si $f_1 \in O(g)$ y $f_2 \in O(h)$, entonces $f_1 \cdot f_2 \in O(g \cdot h)$.

Demostración. A partir de $f_1 \in O(g)$ y $f_2 \in O(h)$, que son nuestras premisas, podemos deducir, por la definición de O , que existen $c_1 \in \mathbb{R}^+$ y $n_1 \in \mathbb{N}$ tales que para todo $n > n_1$ se cumple que

$$f_1(n) \leq c_1 g(n).$$

También de las premisas podemos deducir la existencia de $c_2 \in \mathbb{R}^+$ y $n_2 \in \mathbb{N}$ tales que para todo natural $n > n_2$ se cumple que

$$f_2(n) \leq c_2 h(n).$$

Sea

$$n_0 = \max(n_2, n_1).$$

A partir de lo dicho en el párrafo anterior, podemos decir que para todo natural $n > n_0$ se cumple simultáneamente que:

$$f_1(n) \leq c_1 g(n), \quad f_2(n) \leq c_2 h(n).$$

Si multiplicamos ambas desigualdades obtenemos que:

$$\begin{aligned} f_1(n)f_2(n) &\leq c_1 g(n)c_2 h(n) \\ &= c_1 c_2 g(n)h(n), \end{aligned}$$

por conmutatividad del producto.

Así,

$$f_1(n)f_2(n) \leq (c_1 c_2)g(n)h(n) \quad \text{para todo } n > n_0.$$

Por la definición de O , podemos concluir que

$$f_1 f_2 \in O(g \cdot h).$$

□

Orden de complejidad de MaxSum1

Para finalizar esta sección encontraremos la función de complejidad de los algoritmos MAX-SUM1 utilizando como operación básica esencial la suma, y encontraremos el orden al que pertenecen.

Recordemos que MAXSUM1 era:

Algorithm 1 MAXSUM1(v, n)

```

1:  $sumamax \leftarrow 0$ 
2: for  $i \in \{0, \dots, n-1\}$  do
3:   for  $j \in \{i, \dots, n-1\}$  do
4:      $sumactual \leftarrow 0$ 
5:     for  $k \in \{i, \dots, j\}$  do
6:        $sumactual \leftarrow sumactual + v[k]$ 
7:     end for
8:     if  $sumactual > sumamax$  then
9:        $sumamax \leftarrow sumactual$ 
10:    end if
11:  end for
12: end for
13: return  $sumamax$ 

```

Haciendo el conteo de la operación esencial, tenemos que en la línea 5 hay un ciclo anidado dentro del cual se ejecuta la suma $sumactual + v[k]$, que contamos como una única operación básica, de modo que tenemos:

$$\sum_{k=0}^{n-1} 1.$$

Tal ciclo está anidado en los ciclos de las líneas 2 y 3 respectivamente, lo cual resulta en las siguientes sumas anidadas:

$$\sum_{i=0}^{n-1} \sum_{j=i}^{n-1} \sum_{k=i}^j 1.$$

Como:

$$\sum_{k=i}^j 1 = (j - (i + 1)).$$

Entonces tenemos:

$$\sum_{i=0}^{n-1} \sum_{j=i}^{n-1} (j - (i + 1)).$$

Pero, si expandimos la suma con índice j tenemos:

$$\begin{aligned} \sum_{i=0}^{n-1} ((i - i + 1) + ((i + 1) - i + 1) + \\ ((i + 2) - i + 1) + \cdots + ((n - 1) - i + 1)) \end{aligned}$$

y tras resolver y conmutar los términos de los paréntesis tenemos:

$$\sum_{i=0}^{n-1} (1 + 2 + \cdots + (n - i)) = \sum_{i=0}^{n-1} \sum_{j=1}^{n-i} j.$$

Sabemos que la suma de los primeros n naturales es

$$\frac{n(n + 1)}{2},$$

de modo que nuestra suma anterior se reduce a:

$$\frac{1}{2} \sum_{i=0}^{n-1} (n - i)(n - i + 1).$$

Observemos que si expandimos esta suma obtenemos

$$\frac{1}{2} (n(n + 1) + (n - 1)(n - 1 + 1) + \cdots + 1(1 + 1)),$$

lo cual puede reescribirse como:

$$\begin{aligned} \frac{1}{2} \sum_{i=1}^n i(i + 1) &= \frac{1}{2} \sum_{i=1}^n (i^2 + i) \\ &= \frac{1}{2} \left(\sum_{i=1}^n i^2 \right) + \frac{1}{2} \left(\sum_{i=1}^n i \right) \\ &= \frac{1}{2} \left(\frac{n(n + 1)(2n + 1)}{6} + \frac{n(n + 1)}{2} \right) \\ &= \frac{1}{6} n^3 + \frac{1}{2} n^2 + \frac{1}{3} n. \end{aligned}$$

Entonces nuestra función de complejidad para MAXSUM1 es:

$$f(n) = \frac{1}{6} n^3 + \frac{1}{2} n^2 + \frac{1}{3} n.$$

Ya que la siguiente desigualdad se cumple:

$$\frac{1}{6} n^3 + \frac{1}{2} n^2 + \frac{1}{3} n \leq \frac{1}{6} n^3 + \frac{1}{2} n^3 + \frac{1}{3} n^3 = n^3,$$

con lo que, si consideramos una constante $c = 1$, al resolver la siguiente ecuación:

$$\frac{1}{6}n^3 + \frac{1}{2}n^2 + \frac{1}{3}n = cn^3$$

obtenemos que ambas funciones se intersectan a partir de $n_0 = 1$, por lo que para todo $n > n_0 = 1$ se cumple que

$$f(n) = \frac{1}{6}n^3 + \frac{1}{2}n^2 + \frac{1}{3}n = n^3.$$

Así hemos probado que $f(n) \in O(n^3)$, es decir, el algoritmo MAXSUM1 es de orden cúbico o, dicho de otro modo, queda acotado asintóticamente por una función cúbica.

Ejercicio

Encuentra la función de complejidad $f(n)$ para MAXSUM2 considerando la suma como operación esencial y después demuestra que es de orden cuadrático, es decir, muestra que $f \in O(n^2)$.

Algorithm 2 MAXSUM2(v, n)

```

1: sumamax  $\leftarrow$  0
2: for  $i \in \{0, \dots, n-1\}$  do
3:   sumactual  $\leftarrow$  0
4:   for  $j \in \{i, \dots, n-1\}$  do
5:     sumactual  $\leftarrow$  sumactual +  $v[j]$ 
6:     if sumactual > sumamax then
7:       sumamax  $\leftarrow$  sumactual
8:     end if
9:   end for
10: end for
11: return sumamax

```

Referencias

- [1] María del Carmen Gómez Fuentes and Jorge Cervantes Ojeda. *Introducción al Análisis y al Diseño de Algoritmos*. Universidad Autónoma Metropolitana, jun 2014.